

```
    return result;
}

document.write(exponentByIteration(3, 6) + "<br />");
document.write(exponentByIteration(10, 3) + "<br />");
document.write(exponentByIteration(17, 10) + "<br />");
</script>
```

When you run these programs, the output looks like this:

```
729
1000
2015993900449
```

This is a straightforward calculation that we can easily write with a loop. The downside to using a loop is that the function slows as the exponents get larger: calculating 3^{12} takes twice as long as 3^6 , and 3^{600} takes one hundred times as long as 3^6 . In the next section, we address this by thinking recursively.

Creating a Recursive Exponents Function

Let's think of what a recursive solution for the exponentiation of, say, 3^6 would be. Because of the associative property of multiplication, $3 \times 3 \times 3 \times 3 \times 3 \times 3$ is the same as $(3 \times 3 \times 3) \times (3 \times 3 \times 3)$, which is the same as $(3 \times 3 \times 3)^2$. And since $(3 \times 3 \times 3)$ is the same as 3^3 , we can determine that 3^6 is the same as $(3^3)^2$. This is an example of what mathematics calls the *power rule*: $(a^m)^n = a^{mn}$. Mathematics also gives us the *product rule*: $a^n \times a^m = a^{n+m}$, including $a^n \times a = a^{n+1}$.

We can use these mathematical rules to make an `exponentByRecursion()` function. If `exponentByRecursion(3, 6)` is called, it's the same as `exponentByRecursion(3, 3) * exponentByRecursion(3, 3)`. Of course, we don't actually have to make both `exponentByRecursion(3, 3)` calls: we could just save the return value to a variable and multiply it by itself.

That works for even-numbered exponents, but what about for odd-numbered exponents? If we had to calculate 3^7 , or $3 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3$,